

Trabalho 1 (T1) — Paralelização de uma aplicação com OpenMP

Unidade 03 | Grupos de até 3 integrantes

O trabalho tem três momentos: a apresentação deste enunciado e a formação dos grupos, uma aula de acompanhamento em laboratório em que o professor valida o problema escolhido, e a entrega no Moodle no dia da **apresentação para a turma**. As datas estão no cronograma do Moodle.

1. Objetivo

Pegar uma aplicação sequencial existente, descobrir onde vale a pena paralelizar, instrumentá-la com diretivas OpenMP e **avaliar o desempenho da versão paralela** em uma máquina multicore.

O foco do trabalho não é escrever a aplicação: é a instrumentação e, principalmente, a **análise dos resultados**. Analisar resultados significa explicar o que causou os números obtidos, não repetir em texto o que o gráfico já mostra. Essa distinção é o critério que mais pesa na correção.

Três coisas são postas em uso ao mesmo tempo: as métricas de avaliação de desempenho (speed-up, eficiência, escalabilidade forte e fraca), o método de decidir o que paralelizar (perfil de execução e balanceamento de carga) e as construções do OpenMP (**parallel for**, escopo de variáveis, **reduction**, escalonamento). As seções 4 e 5 detalham o que é exigido de cada uma.

2. Escolha do problema

O grupo escolhe a aplicação. Pode ser um código pronto pego de um repositório, de um livro ou de um artigo, desde que o grupo **saiba explicar o que ele faz**. A escolha precisa ser validada pelo professor na aula de acompanhamento.

Critérios que a aplicação precisa atender

1. **Tempo sequencial suficiente.** A versão sequencial deve levar pelo menos **30 segundos** com a maior entrada usada. Problema que roda em dois segundos não tem grão para paralelizar: o custo de criar a equipe de threads domina o tempo e o speed-up não aparece.
2. **Um eixo de paralelismo identificável.** Deve existir um laço, ou um conjunto de tarefas, que concentre a maior parte do tempo e cujas iterações sejam em boa medida independentes.
3. **Complexidade adequada.** Nem trivial ao ponto de o **parallel for** resolver tudo em uma linha sem nada a analisar, nem tão intrincado que o semestre inteiro não baste. Na dúvida, pergunte no acompanhamento.
4. **Entrada parametrizável.** O tamanho do problema precisa ser controlável por argumento de linha de comando ou constante, porque a escalabilidade fraca (seção 4.2) exige rodar com entradas diferentes.

Uso de ferramentas de IA

Estão liberadas ferramentas de apoio ao desenvolvimento e assistentes de IA. Duas condições: o grupo deve **saber explicar a solução resultante** linha a linha, e o relatório deve indicar **como a ferramenta foi usada** (uma ou duas frases bastam). Na apresentação, qualquer integrante pode ser questionado sobre qualquer parte do código.

3. Ambiente de execução

A execução é na **máquina do grupo** (notebook, desktop ou máquina do laboratório), com o maior número de núcleos disponível. Este trabalho **não** usa o cluster do LAD: ele entra a partir do T2.

Todas as medições devem ser feitas na **mesma máquina**, com ela ociosa (sem navegador, compilação ou outro programa pesado competindo por núcleo).

Descrição da máquina (obrigatória no relatório)

Sem isso é impossível interpretar os resultados apresentados, e em particular identificar o ponto em que o Hyper-Threading (SMT) entra:

- modelo do processador, frequência;
- número de **núcleos físicos** e número de **threads lógicas** (o que `omp_get_num_procs()` devolve);
- tamanho da cache L3 e quantidade de memória RAM;
- sistema operacional, versão do compilador e **flags de compilação** usadas.

Comandos úteis: `lscpu`, `nproc`, `free -h`, `gcc --version`.

Regras de medição

1. **O tempo de referência sequencial (T_1) é o do melhor programa sequencial**, compilado **sem `-fopenmp`**. Não é a versão paralela rodando com uma thread: essa carrega o custo das diretivas e infla o speed-up. Speed-up acima do número de núcleos é quase sempre sinal de T_1 errado.
2. **Mesmas flags de otimização** (`-O2` ou `-O3`) na versão sequencial e na paralela. Comparar sequencial `-O0` com paralelo `-O3` não mede paralelismo, mede o compilador.
3. **Meça a região de interesse**, não o processo inteiro: use `omp_get_wtime()` em volta do trecho computacional. Leitura de arquivo de entrada e escrita do resultado ficam de fora, e o relatório deve dizer o que ficou dentro da medição.
4. **Três execuções por configuração**, no mínimo. Reporte a **mediana** e mencione a variação observada. Uma execução única não distingue efeito real de ruído do sistema.
5. **Fixe o número de threads** por `OMP_NUM_THREADS` (variável de ambiente) ou `omp_set_num_threads()`, e diga no relatório qual dos dois foi usado.

4. Medições exigidas

4.1 Escalabilidade forte

Problema de **tamanho fixo**, número de threads crescente. É a medição que responde “quanto mais rápido fica se eu jogar mais núcleos no mesmo problema”.

Rode com 1, 2 e daí **de 2 em 2** (4, 6, 8, 10, ...) até o número de threads lógicas da máquina. A série precisa **chegar ao total de threads lógicas**, e não parar no número de núcleos físicos: é o trecho acima dos núcleos físicos que revela o efeito do Hyper-Threading. Em uma máquina de 8 núcleos físicos com 16 threads lógicas, por exemplo: 1, 2, 4, 6, 8, 10, 12, 14, 16.

O passo de 2 em 2 é o que dá pontos suficientes para enxergar **onde** a curva satura. Com a série dobrando (1, 2, 4, 8, 16) o joelho da curva cai entre dois pontos medidos e a análise vira suposição.

Threads	Tempo (s)	Speed-up	Eficiência
1 (sequencial, T_1)		1,00	100%
2			
4			
6			
8			
...			
p (threads lógicas)			

Speed-up = T_1 / T_p . Eficiência = speed-up / p.

4.2 Escalabilidade fraca

Trabalho por thread constante: o trabalho total cresce na mesma proporção do número de threads, de modo que cada thread receba sempre a mesma carga. É a medição que responde “consigo resolver um problema maior no mesmo tempo se eu tiver mais núcleos”. No caso ideal, a curva de tempo é uma reta horizontal.

Use a mesma série de threads da escalabilidade forte. O ponto delicado é que **o que cresce proporcionalmente é o trabalho, não o parâmetro de entrada**: com custo proporcional a n^2 , o parâmetro para p threads cresce com a raiz quadrada de p , não com p . Declare no relatório a regra usada e justifique-a pela complexidade do algoritmo.

A coluna abaixo supõe custo proporcional a n ; ajuste-a pela regra do seu algoritmo.

Threads	Tamanho do problema	Tempo (s)	Eficiência
1 (sequencial, T_1)	n		100%
2	$2n$		
4	$4n$		
6	$6n$		
8	$8n$		
...	...		
p (threads lógicas)	$p \cdot n$		

Na escalabilidade fraca a eficiência é T_1 / T_p direto, **sem** dividir por p : o trabalho já cresceu junto com os recursos. A primeira linha da tabela precisa trazer o tempo sequencial e o tamanho da entrada que serve de referência.

4.3 Gráficos

Dois gráficos, no mínimo:

- **speed-up** em função do número de threads, com a **reta do speed-up ideal** ($y = p$) como referência;
- **eficiência** em função do número de threads.

Se houver planilha de apoio disponibilizada no Moodle, use-a: ela já plota as duas curvas no formato esperado. Sempre que possível, coloque várias curvas no mesmo gráfico em vez de gastar espaço com um gráfico por curva. Toda figura precisa de legenda e de referência no texto.

5. O relatório

Duas páginas, em **PDF**. O código-fonte não vai dentro dele: é entregue à parte, como descrito na seção 6.

Página	Conteúdo	Formato
1	Análise: introdução, implementação e discussão dos resultados, com o gráfico de speed-up	Coluna dupla, margens de 1cm, fonte 10
2	Tabelas de tempos, speed-up e eficiência das duas escalabilidades	Coluna simples

Cabeçalho reduzido, com identificação do grupo e do trabalho. Nada de capa: o espaço da primeira página é curto de propósito, e é ele que vale nota.

O que a primeira página precisa cobrir

Organize em subseções curtas (introdução, implementação, análise dos resultados). A proporção esperada é que a **análise ocupe a maior parte** da página.

1. **O problema e seu potencial de paralelização.** O que a aplicação calcula e qual eixo foi escolhido, entre os possíveis. A escolha deve vir de **medição, não de leitura do código**: levante o perfil de execução da versão sequencial com `gprof` ou `perf` e reporte a fração do tempo concentrada no trecho paralelizado, que é também o teto de Amdahl do trabalho.
2. **A modelagem paralela.** Qual é a unidade de trabalho, como ela foi dividida entre as threads, se as unidades de carga são **homogêneas** ou não, e o que atrapalhava o paralelismo (dependências, acumuladores compartilhados, seções críticas) e como foi contornado.
3. **As diretivas usadas.** Quais diretivas e cláusulas OpenMP, onde e por quê, referenciando os trechos por arquivo e linha (por exemplo, `jacobi.c:42`). Se houve otimização (mudança de escopo, troca de `atomic` por `reduction`, ajuste de granularidade), descreva o efeito medido.
4. **Balanceamento de carga.** Como a carga se distribuiu entre as threads e se algum escalonador diferente do padrão (`dynamic`, `guided`, `chunk` ajustado) foi testado. Se `static` foi mantido, justifique. Medição vale mais que suposição aqui.
5. **Análise dos resultados.** A parte central. Explique **o que causou** as curvas obtidas: por que o speed-up satura onde satura, por que a eficiência cai, o que acontece no ponto em que o Hyper-Threading entra (identifique esse ponto no gráfico explicitamente), se há fração sequencial visível (Amdahl) e se a escalabilidade fraca se sustenta melhor que a forte.
6. **Uso de ferramentas de IA**, se houve.

Sobre eficiência: ela é a métrica de custo-benefício, usada quando se paga pelos recursos adicionais. Em uma máquina que o grupo já tem, uma queda de eficiência não é automaticamente um problema. Analise o número, não apenas reporte que ele caiu.

Duas convenções de escrita: prefira texto corrido a listas de itens curtos, que picotam o argumento; e legende toda figura, referenciando-a no texto.

6. O código

Entregue de uma das duas formas, à escolha do grupo:

- um **arquivo .zip** enviado no Moodle junto do PDF; ou
- um **link para repositório git** (GitHub, GitLab), informado no cabeçalho do relatório.

Se optar pelo repositório, ele precisa estar **público** e o relatório deve indicar o **commit ou tag da entrega**. Um repositório privado não pode ser corrigido, e um que continue mudando depois do prazo deixa de corresponder ao que foi medido.

Em qualquer das duas formas, o que é entregue precisa ser suficiente para **reproduzir os resultados apresentados**: os fontes, o `Makefile` ou a linha de compilação usada, o gerador ou o arquivo de entrada, e um `README` curto com o comando exato de cada medição. Se faltar alguma dessas peças, o resultado não é reproduzível.

Nomes de variáveis claros e comentários nos pontos não óbvios (por que uma variável é `firstprivate`, por que aquele `chunk`) contam na avaliação.

7. Apresentação

No dia da entrega, para a turma, com **participação de todos os integrantes**. O tempo é curto: priorize a modelagem paralela e a análise dos resultados. Espere perguntas sobre decisões específicas do código.

8. Avaliação

A nota considera os itens abaixo. Cada um é avaliado em quatro níveis: atingiu plenamente, atingiu parcialmente, quase não atingiu, não atingiu.

#	Item
1	O relatório está bem escrito e no formato solicitado
2	O problema escolhido é interessante e adequado para a aplicação de paralelismo
3	O problema tem complexidade adequada para a proposta do trabalho
4	O grupo usou corretamente as diretivas OpenMP e explicou no relatório como foram usadas
5	O código foi entregue de forma acessível, permitindo reproduzir os resultados apresentados
6	A modelagem paralela está detalhada: escolha do laço, eixo paralelizado, homogeneidade das unidades de carga
7	A questão do balanceamento de carga foi abordada, incluindo a pertinência de outro escalonamento
8	O uso de Hyper-Threading foi avaliado, com o ponto correspondente indicado no gráfico
9	As escalabilidades forte e fraca foram plotadas e a eficiência calculada
10	Boa parte do relatório foi dedicada a uma análise aprofundada dos resultados
11	Participação do grupo nas aulas de acompanhamento no laboratório
12	Apresentação do trabalho com participação de todos os integrantes

9. Entrega

No Moodle, até a data indicada no cronograma: o relatório em `.pdf` e, se o grupo optou pelo `.zip`, o código junto. Quem entregar o código por repositório git envia só o PDF, com o link e o commit no cabeçalho. Um envio por grupo, com os nomes dos integrantes no cabeçalho.